

## APIs de IA — Conceptos Clave y Primeras Llamadas

Ejercicio de Clase · Ejercicio 2 de 3 · Bloques 4.3 — 4.5 — 4.6 — 4.7

|                                                                                                                 |  |
|-----------------------------------------------------------------------------------------------------------------|--|
| Nombre del alumno/a                                                                                             |  |
| Fecha                                                                                                           |  |
|  Enlace GitHub del repositorio |  |

 **Instrucciones:** Aplica los conceptos clave vistos en clase. Este ejercicio combina teoría y práctica. Analiza código, diseña prompts y toma decisiones técnicas. ¡No hay que escribir código desde cero, pero sí entender y razonar sobre él!

### BLOQUE 1 — Conceptos clave aplicados (4.3)

Vimos que tokens, context window, temperature y system prompt son los parámetros clave de cualquier llamada a una API de IA. Para cada escenario, identifica qué parámetro hay que ajustar y cómo:

| Escenario / Problema                                                                             | ¿Qué parámetro ajustas? | ¿Cómo lo ajustas? |
|--------------------------------------------------------------------------------------------------|-------------------------|-------------------|
| Tu chatbot genera respuestas demasiado largas y consumes muchos tokens.                          |                         |                   |
| La IA responde de forma demasiado creativa y sus respuestas varían mucho ante la misma pregunta. |                         |                   |
| Tu modelo 'olvida' el principio de la conversación cuando el historial es muy largo.             |                         |                   |
| Quieres que la IA actúe siempre como un asistente técnico experto en Python.                     |                         |                   |
| Los tests generados por la IA no siguen el formato de tu proyecto (pytest, sin clases).          |                         |                   |
| Necesitas buscar documentos por significado, no por palabras exactas.                            |                         |                   |

### 1.2 — System Prompt: diseña el comportamiento

Diseña un system prompt profesional para cada uno de estos asistentes. Recuerda que debe incluir: rol, tono, reglas y formato de respuesta:

| Asistente                                                               | Tu system prompt |
|-------------------------------------------------------------------------|------------------|
| Un asistente técnico experto en APIs de IA para desarrolladores junior. |                  |
| Un clasificador de tickets de soporte que responde SOLO en JSON.        |                  |
| Un tutor de programación que explica conceptos con analogías simples.   |                  |

## 1.3 — Few-shot Learning: enseña con ejemplos

Vimos que el few-shot learning consiste en incluir ejemplos en el prompt para que el modelo aprenda el patrón. Completa la tabla indicando los ejemplos que añadirías al prompt y qué patrón quieres que aprenda:

| Tarea que quieres que la IA haga                                         | 2-3 ejemplos que incluirías en el prompt | Patrón que aprende el modelo |
|--------------------------------------------------------------------------|------------------------------------------|------------------------------|
| Clasificar reseñas de productos como POSITIVO / NEUTRO / NEGATIVO.       |                                          |                              |
| Extraer el nombre y el email de textos en lenguaje natural.              |                                          |                              |
| Traducir frases técnicas de inglés a español manteniendo el tono formal. |                                          |                              |

## BLOQUE 2 — Comparativa de precios y optimización de costes (4.5)

En clase vimos que el coste de las APIs de IA se calcula en tokens. Responde las siguientes preguntas:

| Pregunta                                                                 | Tu respuesta |
|--------------------------------------------------------------------------|--------------|
| ¿Qué modelo recomendarías para una app educativa de prototipo y por qué? |              |

|                                                                                       |  |
|---------------------------------------------------------------------------------------|--|
| ¿Cuál es el modelo más económico de Anthropic Claude y para qué es ideal?             |  |
| ¿Qué modelo de Google destacó en visión + texto como alternativa a GPT-4o?            |  |
| ¿Por qué Gemini 1.5 Flash es ideal para apps que requieren respuestas en tiempo real? |  |
| Nombra 3 estrategias concretas para reducir el coste de llamadas a la API.            |  |

## 2.2 — Calcula el coste real

Usando lo aprendido en clase, calcula el coste aproximado de las siguientes aplicaciones. Muestra tu razonamiento:

| Aplicación                                                                                                                                              | Cálculo paso a paso | Coste estimado |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|----------------|
| Chatbot que recibe 100 consultas/día. Cada consulta: 200 tokens input + 300 tokens output. Modelo: GPT-4o-mini (\$0.002/1K tokens).                     |                     |                |
| Sistema que resume artículos. Procesa 50 artículos/día de 1.500 tokens cada uno, genera resúmenes de 300 tokens. Modelo: Gemini Flash (muy bajo coste). |                     |                |

## BLOQUE 3 — Análisis de llamadas a la API (4.6 — 4.7)

Analiza el siguiente código de llamada a la API de OpenAI en Python. Responde las preguntas sin escribir código nuevo:

```
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))

response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[
        {'role': 'system', 'content': 'Eres un asistente experto en APIs.'},
        {'role': 'user', 'content': '¿Qué es un token?'}
    ],
    temperature=0.3,
    max_tokens=500
)

print(response.choices[0].message.content)
```

| Pregunta sobre el código                                                                             | Tu respuesta |
|------------------------------------------------------------------------------------------------------|--------------|
| ¿Por qué se usa <code>os.getenv('OPENAI_API_KEY')</code> en lugar de escribir la clave directamente? |              |
| ¿Qué hace el mensaje con role 'system'?<br>¿Por qué es diferente al 'user'?                          |              |
| ¿Qué efecto tiene <code>temperature=0.3</code> en las respuestas del modelo?                         |              |
| ¿Qué significa <code>max_tokens=500</code> ? ¿Afecta al input o al output?                           |              |
| ¿Qué devuelve <code>response.choices[0].message.content</code> ?                                     |              |
| ¿Cómo añadirías el historial de conversación para que el modelo recuerde mensajes anteriores?        |              |

## 3.2 — JavaScript vs Python: diferencias prácticas

En clase vimos la misma llamada en Node.js y Python. Completa la tabla comparativa:

| Aspecto                                                   | JavaScript (Node.js) | Python |
|-----------------------------------------------------------|----------------------|--------|
| Forma de importar el cliente OpenAI                       |                      |        |
| Gestión de variables de entorno                           |                      |        |
| ¿Requiere <code>async/await</code> ?                      |                      |        |
| Extracción de la respuesta del modelo                     |                      |        |
| ¿Para qué entorno es más habitual?                        |                      |        |
| ¿Se usa el mismo endpoint <code>chat.completions</code> ? |                      |        |

## BLOQUE 4 — Integrando APIs de IA con lo que ya sabes (Pipeline + Modularización)

Ya conoces GitHub Actions y la modularización de código. Ahora conéctalo con las APIs de IA:

| Pregunta de integración                                                                          | Tu respuesta |
|--------------------------------------------------------------------------------------------------|--------------|
| ¿Dónde guardarías la API key de OpenAI en un repositorio de GitHub? ¿Por qué no en el código?    |              |
| ¿Cómo pasarías la API key de GitHub Secrets a tu script Python en un workflow de GitHub Actions? |              |
| Si modularizas tu app con un módulo separado para las llamadas a la API, ¿qué ventajas tiene?    |              |
| ¿Qué problema puede ocurrir si tu pipeline ejecuta la llamada a la API en un bucle sin límite?   |              |

## BLOQUE 5 — Pregunta de síntesis

*Diseña una aplicación real que use una API de IA. Indica: (1) qué problema resuelve, (2) qué proveedor y modelo elegirías, (3) qué parámetros clave usarías (temperature, max\_tokens, system prompt), (4) cómo integrarías la API key de forma segura en un repositorio con pipeline CI/CD. Justifica cada decisión:*